CPE100L – Digital Logic Design I

Laboratory topic:	7-segment display and code converters	**Lab #:**	9
Author:	Hassan Adam		
Class mode:	On Campus		

1. Goals

Learn how to use the DE0-CV's 7-segment displays. Design a binary-to-7-segment decoder yourself with Karnaugh maps, extend it from BCD to full hexadecimal, drive two digits at once, and use it to display the result of the 2-bit ALU from Lab 8. The second half of the lab treats the decoder as one example of a code converter and adds two more: binary-to-Gray and BCD-to-excess-3.

2. Textbook References

Fundamentals of Logic Design, Charles H. Roth, Jr. and Larry L. Kinney.

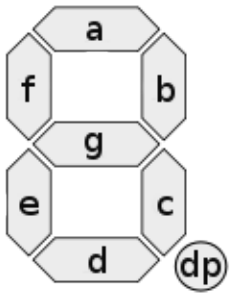
Unit	Topic	Key Sections
Unit 1	Binary codes: BCD, excess-3, Gray code	Sec 1.5, Table 1-2
Unit 5	Karnaugh maps with don't-cares	Sec 5.3–5.4
Unit 7	Two-level multiple-output circuits (sharing product terms between outputs)	Sec 7.6
Unit 8	Multi-output circuit design; the seven-segment indicator	Figure 8-15 and design problems 8. A, 8. B, 8. F (pp. 246–248)
Unit 9	Decoders (the 7-segment decoder as a decoder plus OR gates)	Sec 9.4

Table 1. Textbook sections used in this lab

3. Background

3.1. The 7-segment display

A binary-coded number can be shown on a seven-segment display through a decoder circuit that converts the number into seven segment-control signals, labeled a through g. (The textbook's Figure 8-15 numbers the segments 1–7; the letters a–g used here are the standard industry labels for the same segments.) A dedicated decoder IC such as the 7447 performs this conversion in hardware; in this lab, you will design the decoder logic yourself using Karnaugh maps (Roth Unit 5), as in design problem 8. A.



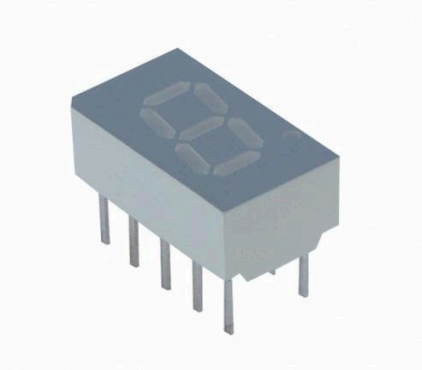


Figure 1. 7-segment display: (top) segment labels; (bottom) photo of a display

The target truth table is given below. The decoder inputs are labeled d3 d2 d1 d0 ("d" for display) to avoid confusion with ALU operand b. Outputs are active-low: a 0 lights the segment, matching the DE0-CV's common-anode displays. Rows 6 and 9 follow the 7447 convention: 6 is shown without segment a, and 9 without segment d.

Digit	d3 d2 d1 d0	a	b	c	d	e	f	g
0	0 0 0 0	0	0	0	0	0	0	1
1	0 0 0 1	1	0	0	1	1	1	1
2	0 0 1 0	0	0	1	0	0	1	0
3	0 0 1 1	0	0	0	0	1	1	0
4	0 1 0 0	1	0	0	1	1	0	0
5	0 1 0 1	0	1	0	0	1	0	0
6	0 1 1 0	0	1	0	0	0	0	0
7	0 1 1 1	0	0	0	1	1	1	1
8	1 0 0 0	0	0	0	0	0	0	0
9	1 0 0 1	0	0	0	0	1	0	0

Table 2. BCD-to-7-segment decoder truth table (active-low outputs)

Input combinations 10 through 15 cannot occur for a BCD digit and are don't-cares — use them to simplify your Karnaugh maps (Roth Sec 5.3). The decoder is a multi-output circuit: seven functions of the same four variables. Roth Sec 7.6 (two-level multiple-output design) shows that the cheapest design often shares product terms between outputs, so after minimizing each output separately, look for terms that appear in more than one map.

There are two types of 7-segment displays: common anode (a logic 0 lights a segment) and common cathode (a logic 1 lights a segment). The DE0-CV's displays HEX0–HEX5 are common anode, which is why Table 2 is written active-low. In Pin Planner, each display appears as HEX0[0]–HEX0[6], where index 0 is segment a and index 6 is segment g.

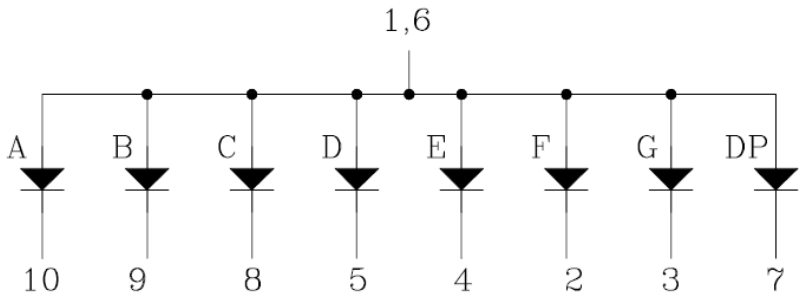


Figure 2. Common-anode 7-segment display circuit

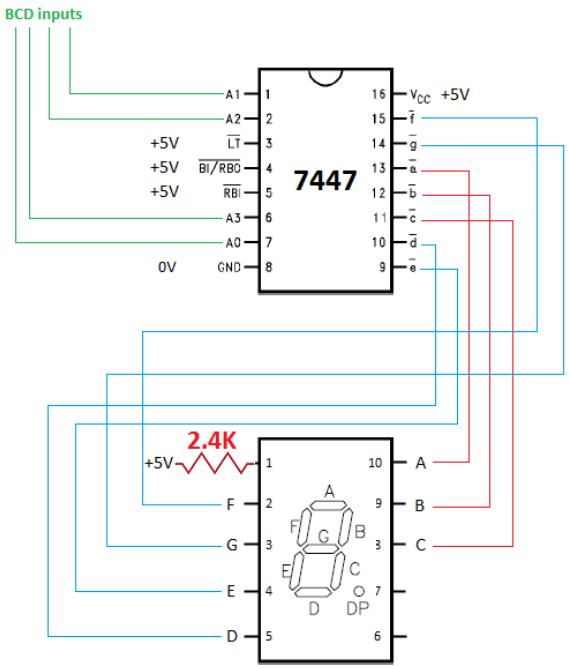


Figure 3. Example of decoding a BCD number using the 7447 IC

3.2. HEXADECIMAL DIGITS

The DE0-CV displays are used with 4-bit values throughout the rest of the course, so the decoder must also show 10–15 as A, b, C, d, E, F (lower-case b and d so they are not confused with 8 and 0). This removes the don't-cares of Table 2 and gives the full 16-row table you will complete in the prelab.

Digit	d3 d2 d1 d0	a	b	c	d	e	f	g
A	1 0 1 0	0	0	0	1	0	0	0
b	1 0 1 1	1	1	0	0	0	0	0
C	1 1 0 0	0	1	1	0	0	0	1
d	1 1 0 1	1	0	0	0	0	1	0
E	1 1 1 0	0	1	1	0	0	0	0
F	1 1 1 1	0	1	1	1	0	0	0

Table 3. Hexadecimal extension of the decoder (active-low outputs)

3.3. Code converters

A code converter is a combinational circuit that maps one binary code onto another; the 7-segment decoder is one (binary to segment code). Two more from Roth Sec 1.5 are used in this lab:

- **Gray code.** Successive values differ in exactly one bit, which prevents false intermediate readings in mechanical encoders and matters for K-map ordering. For a 4-bit binary number b3 b2 b1 b0 the Gray code is g3 = b3, g2 = b3 ⊕ b2, g1 = b2 ⊕ b1, g0 = b1 ⊕ b0.
- **Excess-3.** A BCD digit plus 3, once used to simplify decimal subtraction. Its converter is a 4-input, 4-output function with the same six don't-cares as Table 2 (Roth design problem 8. B drives a seven-segment display from excess-3).

Decimal	Binary / BCD	Gray	Excess-3
0	0000	0000	0011
1	0001	0001	0100
2	0010	0011	0101
3	0011	0010	0110
4	0100	0110	0111
5	0101	0111	1000
6	0110	0101	1001
7	0111	0100	1010
8	1000	1100	1011
9	1001	1101	1100

Table 4. Binary, Gray, and excess-3 codes for 0–9

Note on the pushbuttons. The DE0-CV pushbuttons KEY0–KEY3 are active-low: a released button reads logic 1 and a pressed button reads logic 0. Either account for this when testing, or place an inverter after each KEY input so that pressing means logic 1. State in your report which approach you used.

4. Prelab

1. BCD decoder. Using Karnaugh maps, derive a minimized expression for each of the seven outputs a–g of Table 2. Treat input combinations 10–15 as don't-cares. Show the K-map and the minimization for every output, and mark any product term shared between two or more outputs.
2. Hex decoder. Combine Tables 2 and 3 into one 16-row table with no don't-cares and minimize all seven outputs again. Draw the decoder in Quartus and save it as a symbol (hex7seg). Compare the number of product terms with item 1.
3. Two digits. Plan how a 3-bit value 0–7 and a letter code A/o/E/d (for AND, OR, XOR, ADD) will be shown on HEX0 and HEX1 in Experiment 3; write the segment pattern for each of the four letters.
4. Gray code and excess-3. Derive the four Gray-code equations from Table 4 (they need no K-map) and, with K-maps and don't-cares, the four excess-3 equations. Draw both converters in Quartus.
5. Prelab deliveries. Include in the prelab document:
    - K-maps, minimized expressions, and the minimization process for all seven outputs, for both the BCD and the hex decoder
    - The hex7seg schematic and a ModelSim waveform showing all 16 codes (d0–d3 clocks of 20, 40, 80, 160 ns)
    - The letter patterns of item 3
    - Gray and excess-3 equations and schematics
    - Answer: Does Table 2 represent a common-anode or a common-cathode display? How can you tell?

5. Lab Experiments

Set the ModelSim path to *modelsim_ase\win32aloem* in every new project (Lab 4, Section 6); a license error means the path points to *modelsim_ae*.

Record *Total logic elements* from the Compilation Report for every circuit. Switch, LED, and display assignments:

Experiment	Inputs	Outputs	PIN names (fill in from the pin table)
1 3-bit number	d2 d1 d0 = SW2–SW0 (d3 = 0)	HEX0[6..0] (active-low)	_____
2 Hex decoder	d3–d0 = SW3–SW0	HEX0	_____
3 ALU + letter	a1 a0 = KEY3–KEY2, b1 b0 = KEY1–KEY0 (active-low), S1 S0 = SW1–SW0	result HEX0, operation letter HEX1	_____
4 Converters	b3–b0 = SW3–SW0	Gray LEDR3–LEDR0, input HEX2, excess-3 HEX3	_____

Table 5. Switch, LED and display assignment for this lab (Experiments 1–2 share one project)

In Pin Planner, each display is HEX0[0] … HEX0[6], where [0] is segment a and [6] is segment g. A 0 lights the segment.

Clock d0, d1, d2, d3 at 20, 40, 80, 160 ns so all 16 codes appear and read the seven outputs as a 7-bit bus.

5.1. Experiment 1

1. Implement a circuit that displays a 3-bit binary number (0–7) on the DE0-CV's HEX0 display, using the decoder logic from your prelab K-maps with input d3 tied to 0.
2. Connect the three bits of the number to switches SW0 (d0), SW1 (d1), and SW2 (d2).
3. Verify all eight values 0–7 against Table 2.
4. Demonstrate the circuit to the TA.

5.2. Experiment 2

1. Replace the decoder with hex7seg and connect d3 to SW3.
2. Verify all sixteen codes 0–F on HEX0. Record the LE count and compare it with the BCD-only decoder of Experiment 1.

Record the Experiment 2 results in Table 6:

d3 d2 d1 d0	Expected character	HEX0 shows	OK?
0000	0		
0001	1		
0010	2		
0011	3		
0100	4		
0101	5		
0110	6		
0111	7		
1000	8		
1001	9		
1010	A		
1011	b		
1100	C		
1101	d		

d3 d2 d1 d0	Expected character	HEX0 shows	OK?
1110	E		
1111	F		

Table 6. Hex decoder verification (fill in)

3. Demonstrate to the TA.

5.3. Experiment 3

1. Place the alu2 symbol from Lab 8 and two hex7seg symbols in a new top-level schematic. Operands a1 a0 on KEY3–KEY2, b1 b0 on KEY1–KEY0 (active-low), select S1 S0 on SW1–SW0.
2. Connect the ALU result y2 y1 y0 to the HEX0 decoder inputs d2 d1 d0 (d3 = 0), so the result is displayed on HEX0.
3. Drive HEX1 with the letter for the current operation: build a small decoder from S1 S0 to the four-segment patterns of your prelab item 3 (or use mux4 symbols on each segment line, one per segment, with constant inputs).
4. Verify that ADD displays sums 0–6 correctly and that the logic operations display their 2-bit results (0–3) with the correct letter beside them. Demonstrate to the TA.

5.4. Experiment 4

1. Implement the binary-to-Gray converter: binary input on SW3–SW0, Gray output on LEDR3–LEDR0.
2. Step the switches through 0, 1, 2, …, 15 in order and confirm that exactly one LED changes between consecutive values.

Record the Gray-code results in Table 7 and mark which LED changed from the previous row:

b3 b2 b1 b0	Expected Gray g3 g2 g1 g0	LEDR3–LEDR0	Bit that changed
0000	0000		
0001	0001		
0010	0011		
0011	0010		
0100	0110		
0101	0111		
0110	0101		
0111	0100		
1000	1100		
1001	1101		
1010	1111		
1011	1110		
1100	1010		
1101	1011		
1110	1001		
1111	1000		

Table 7. Binary-to-Gray verification (fill in)

3. Implement the BCD-to-excess-3 converter in the same project with the same input switches. Show its 4-bit output on HEX3 using a third hex7seg, and the binary input on HEX2, so that the two displays read, e.g., "5" and "8" side by side.
4. Verify Table 4 and demonstrate to the TA.

6. What to Turn In (Postlab)

1. Theory of Operation

   [1 pt] Please indicate how many hours you spent on this lab. You will get one point for listing your time on each lab. Please note any bugs you found in this lab manual or suggestions to improve the lab.

2. [6 pts total] Experimental Results
   a) Experiment 1
      - [0.5 pt] Screenshot of the Quartus schematic
      - [0.5 pt] Photo of the DE0-CV showing a value on HEX0, with the switch positions visible
      - [0.5 pt] Eight-row table of switch value versus displayed digit

   b) Experiment 2
      - [0.5 pt] hex7seg schematic and ModelSim waveform of the 16 codes
      - [0.5 pt] Photo of HEX0 showing a letter (A–F)
      - [0.5 pt] LE count comparison, BCD versus hex decoder, with one sentence on why they differ

   c) Experiment 3
      - [0.5 pt] Screenshot of the top level showing ALU, letter decoder and both hex7seg symbols
      - [0.5 pt] Photo of the DE0-CV displaying an ADD result greater than 3 with the "d" beside it
      - [0.5 pt] Brief description of how the letter decoder was built

   d) Experiment 4
      - [0.5 pt] Schematics of both converters
      - [0.5 pt] Photo of HEX2/HEX3 showing a BCD digit and its excess-3 code
      - [0.5 pt] Table of Gray outputs for 0–15 with the changed bit marked on each row

3. [2 pts total] Questions
   a) [0.5 pt] Why does the 2-bit ADD operation's largest value still fit on a single 7-segment digit, and what would you need to display a 3-bit ALU's sum?
   b) [0.5 pt] Give an example of a product term you were able to share between two decoder outputs, and say how many gates it saved (Roth Sec 7.6).
   c) [0.5 pt] Why does adding the six hexadecimal rows generally increase the cost of the decoder even though they add no new outputs?
   d) [0.5 pt] Why is Gray code used to label the rows and columns of a Karnaugh map?

4. [1 pt] Conclusions

   Summarize what you learned, challenges encountered during the lab, how they were resolved, and any additional observations.

5. Attachments

   Zip your projects and provide a link to the zip file on Google Drive / Dropbox, etc. List of attachments to deliver:
   - 7-segment decoder project (with hex7seg symbol)
   - 2-bit ALU with 7-segment display project (Experiment 3)
   - Code converters project (Experiment 4)

References:

1. C. H. Roth, Jr. and L. L. Kinney, *Fundamentals of Logic Design*, Enhanced Ed. Cengage Learning, 2021.
2. DE0 pin table: http://faculty.unlv.edu/eelabs/docs/guides/DE0_Pin_Table.pdf
3. *Altera DE0 Introduction / User Manual*: http://faculty.unlv.edu/eelabs/docs/guides/DE0_CV_User_Manual.pdf
4. Datasheets of 7400 series chips (7447): http://faculty.unlv.edu/eelabs/index.html?navi=main_icdatasheets
5. *Simulating Design in Quartus 13.1* (updated October 2026) — posted on WebCampus